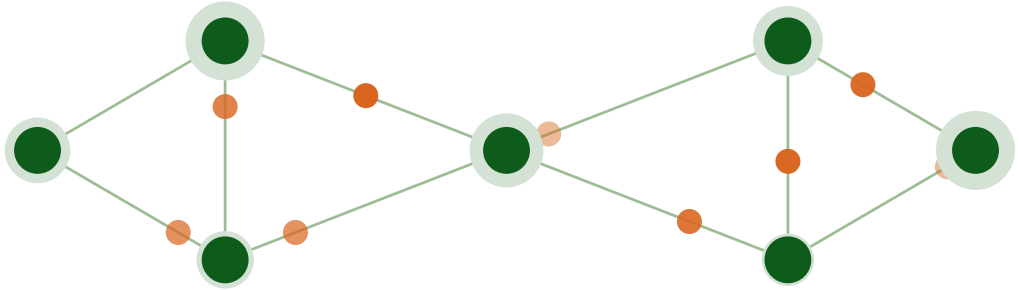
SupNum

Les origines du NoSQL

Mamiya Megueye

Bases de données avancées — L2 IDS



Objectifs du cours



Plongez dans l'univers des **bases de données NoSQL**, incontournables dans les architectures modernes. Vous allez :

- **comprendre** pourquoi les bases relationnelles ne suffisent plus face aux besoins actuels ;
- **explorer** les types de bases NoSQL : documents, colonnes, graphes, clés-valeurs ;
- **découvrir** les fondements de **MongoDB**, la star des bases orientées documents.

L'objectif

Vous rendre **autonomes** pour comprendre, manipuler et intégrer des bases NoSQL dans vos projets, avec des outils professionnels : **MongoDB Compass** et **MongoDB Atlas**.

Vue d'ensemble du module

- **Chapitre 1 :** Les origines du NoSQL
- **Chapitre 2 :** Distribuer les données à l'échelle
- **Chapitre 3 :** Choisir sa base de données NoSQL
- **Chapitre 4 :** Découvrir le fonctionnement de MongoDB
- **Chapitre 5 :** Interroger MongoDB et indexer les données

Plan du cours

1. Introduction
   - Ce que vous savez déjà
   - C'est quoi le NoSQL ?
   - MySQL et MongoDB : un même exemple
2. Limites des bases de données relationnelles
   - Cinq limites
3. Modèles transactionnels : ACID et BASE
   - Le modèle ACID
   - Le modèle BASE

Pour commencer : à vous de réfléchir

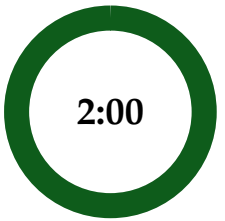
Question 1

Quelles applications utilisez-vous tous les jours ? Selon vous, où et comment sont stockées vos données ?

Question 2

WhatsApp traite chaque jour des milliards de messages dans le monde. Quelles difficultés rencontrerait une seule base MySQL, installée sur un seul serveur ?

Consigne : réfléchissez 2 minutes, puis partagez vos idées avec la classe.



cliquer pour lancer

Introduction

Rappel : le modèle relationnel

Exemple : l'application de scolarité d'une université.

etudiants

id_etudiant	nom	filiere	niveau
1	Ahmed	IDS	L2
2	Fatima	IDS	L2

inscriptions

id_etudiant	*code_cours*	note
1	BDA	14
2	BDA	16

Question

Si je veux récupérer le **nom** de chaque étudiant avec sa **note en BDA**, qu'est-ce que je dois faire ?

```
SELECT e.nom, i.note
FROM etudiants e
JOIN inscriptions i
  ON e.id_etudiant = i.id_etudiant
WHERE i.code_cours = 'BDA';
```

- **Table** : un type d'objet (`etudiants`)
- **Ligne** : un enregistrement (Fatima)
- **Colonne** : un attribut typé (`nom`)
- **Clé primaire** (soulignée) : identifie chaque ligne
- **Clé étrangère** (*italique*) : relie deux tables
- **Jointure** : recombine les tables dans une requête
- **Schéma** : la structure, définie à l'avance

Deux façons d'écrire la même jointure

Les deux requêtes renvoient **le même résultat** : le nom de chaque étudiant avec sa note en BDA.

Avec `JOIN … ON` (SQL-92)

```sql
SELECT e.nom, i.note
FROM etudiants e
JOIN inscriptions i
  ON e.id_etudiant = i.id_etudiant
WHERE i.code_cours = 'BDA';
```

Jointure implicite (SQL-89)

```sql
SELECT e.nom, i.note
FROM etudiants e, inscriptions i
WHERE e.id_etudiant = i.id_etudiant
  AND i.code_cours = 'BDA';
```

Et si notre application grandissait ?

L'université lance une application pour tous ses étudiants, puis l'ouvre à tout le pays.

1. Les **profils deviennent très variés** : compétences, projets, liens GitHub, stages… → **documents**
2. Des **milliers de connexions par minute** : il faut retrouver une session instantanément. → **clé-valeur**
3. Des **capteurs** dans les salles envoient une mesure chaque seconde : écritures massives. → **colonnes larges**
4. On veut suggérer des « **amis d'amis** » : des relations en cascade. → **graphes**

Question

Une seule base relationnelle est-elle la meilleure réponse à **tous** ces besoins ?

C'est quoi le NoSQL ?

Définition

Le **NoSQL** (*Not Only SQL*) désigne une famille de bases de données **non relationnelles**, conçues pour répondre aux défis de la **gestion de données à grande échelle**.

- *Not Only SQL* : « pas seulement SQL », et non « pas de SQL ».
- Les données ne sont plus forcément en tables : **documents**, **clés-valeurs**, **colonnes larges**, **graphes**.
- Le NoSQL **assouplit certaines contraintes** du modèle relationnel (schéma fixe, jointures).
- Le terme s'est diffusé vers **2009**, avec l'essor des grands services web.

Remarque

Le NoSQL ne remplace pas le relationnel : les deux coexistent selon les besoins.

MySQL vs MongoDB (1/2) : deux façons de modéliser

Ahmed et Fatima. **Fatima a plusieurs compétences et deux adresses.**

MySQL : 3 tables reliées

etudiants

id	nom
1	Ahmed
2	Fatima

competences

id_etudiant	competence
1	Java
2	Python
2	SQL
2	Git

adresses

id_etudiant	type	ville
1	domicile	Rosso
2	domicile	Nouakchott
2	stage	Nouadhibou

MongoDB : 1 document par étudiant

```
{ "_id": 2,
  "nom": "Fatima",
  "competences": ["Python", "SQL", "Git"],
  "adresses": [
    { "type": "domicile",
      "ville": "Nouakchott" },
    { "type": "stage",
      "ville": "Nouadhibou" } ] }
```

Ce qui change

MySQL **répartit** les données dans des tables reliées par des clés étrangères ; MongoDB les **regroupe** dans un document, avec tableaux et sous-documents.

MySQL vs MongoDB (2/2) : interroger et nommer

Question : quelles sont les compétences de Fatima ?

MySQL (SQL)

```sql
SELECT c.competence
FROM etudiants e
JOIN competences c
  ON c.id_etudiant = e.id
WHERE e.nom = 'Fatima';
```

MongoDB

```
db.etudiants.find(
  { nom: "Fatima" },
  { competences: 1 }
)
```

Relationnel (MySQL)	Documents (MongoDB)	
Table	Collection	`etudiants`
Ligne	Document	la fiche de Fatima
Colonne	Champ	`nom`
Clé primaire	`_id`	`2`
Jointure	Données imbriquées (ou `$lookup`)	`competences`

Limites des bases de données relationnelles

Limites des bases relationnelles

1. **Passage à l'échelle et distribution complexes** : répartir les données et les opérations sur plusieurs serveurs peut demander une architecture complexe.
   Exemple : une application utilisée par des millions d'étudiants.

2. **Schéma moins flexible** : lorsque les données changent souvent ou sont très variées, faire évoluer le modèle relationnel peut devenir contraignant.
   Exemple : des profils étudiants avec des informations différentes.

3. **Jointures coûteuses à grande échelle** : certaines jointures sur de gros volumes ou des données distribuées demandent beaucoup de ressources.
   Exemple : un réseau social avec des millions de publications et de « J'aime ».

4. **Coordination des transactions** : garantir des transactions strictes entre plusieurs serveurs peut augmenter les délais.
   Exemple : un virement bancaire traité par plusieurs serveurs.

5. **Relations en cascade** : parcourir de nombreuses relations peut devenir complexe.
   Exemple : retrouver les amis d'amis d'un étudiant.

À vous de réfléchir : les limites

Question 3

Instagram ajoute de nouvelles fonctionnalités presque chaque mois (stories, réels, sondages…). Laquelle des cinq limites du relationnel cela pose-t-il en premier ?

Question 4

Parmi les cinq limites, laquelle vous semble la plus gênante pour un **réseau social** ? Et pour une **banque** ? Justifiez.

Modèles transactionnels : ACID et BASE

Le modèle ACID

Une **transaction** regroupe plusieurs opérations qui doivent former un tout. Le modèle **ACID** garantit son bon déroulement.

Exemple : un virement bancaire.

Propriété ACID	Sur le virement
A*tomicité* : la transaction s'exécute entièrement ou pas du tout	Le débit et le crédit passent ensemble, ou aucun des deux
C*ohérence* : la base passe d'un état valide à un autre état valide (règles respectées)	Le total d'argent est conservé ; aucun solde ne devient négatif si la règle l'interdit
I*solation* : les transactions simultanées ne voient pas les étapes intermédiaires des autres	Deux retraits simultanés sur un compte commun ne lisent pas un solde « à moitié modifié »
D*urabilité* : une transaction validée est conservée, même après une panne	Le virement confirmé reste enregistré après le redémarrage

À noter : ACID n'est pas réservé au SQL : MongoDB ou Neo4j proposent aussi des transactions ACID.

À vous de réfléchir : ACID

Question 5 — Mise en situation

Vous faites un virement de 5 000 MRU vers un ami. Au milieu de l'opération, le courant coupe : votre compte est débité, mais le sien n'est pas encore crédité.

- Que doit faire la base de données ?
- Quelle propriété ACID est en jeu ?

Question 6

Deux personnes retirent de l'argent **au même moment** sur un compte commun. Quelle propriété ACID protège le solde ?

Le virement interrompu, pas à pas

La transaction

```
START TRANSACTION;
UPDATE compte SET solde = solde - 5000
  WHERE titulaire = 'vous';
UPDATE compte SET solde = solde + 5000
  WHERE titulaire = 'ami';
COMMIT;
```

Votre compte
20 000 MRU

Compte de l'ami
3 000 MRU

Pas de `COMMIT` ⇒ `ROLLBACK`
les deux comptes reviennent à leur état initial

Ce que chaque propriété garantit ici

- **A** : le débit seul est annulé, rien n'est perdu.
- **C** : le total reste 23 000 MRU avant et après.
- **I** : pendant la transaction, personne n'a lu le solde provisoire de 15 000 MRU.
- **D** : si la coupure était survenue *après* le `COMMIT`, le virement serait conservé.

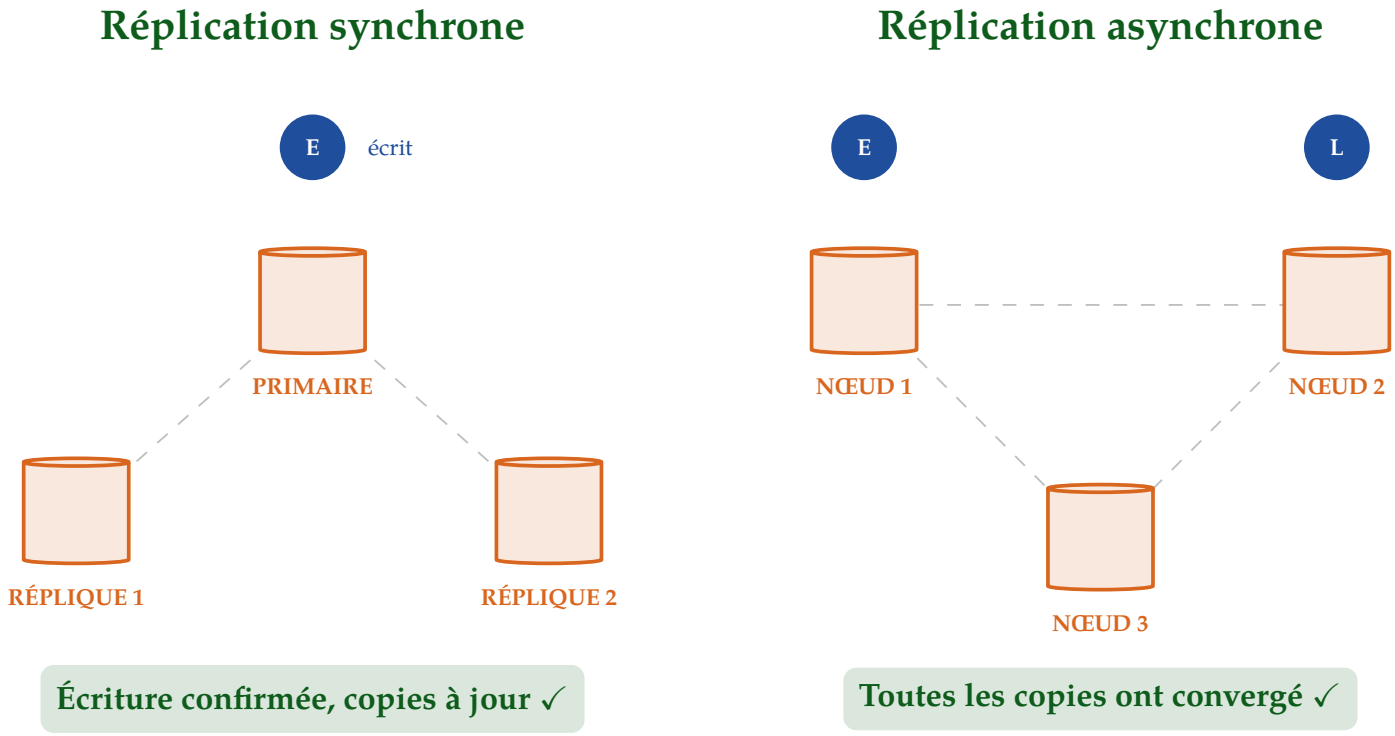
Le modèle BASE

Exemple : le compteur de « J'aime » d'une photo, copié sur plusieurs serveurs.

Propriété BASE	**Sur le compteur de « J'aime »**
BA — *Basically Available* : le système répond toujours, même sous forte charge	Un nombre de « J'aime » s'affiche toujours, même pendant un pic de trafic
S — *Soft state* : l'état peut évoluer sans nouvelle requête, le temps que les copies se synchronisent	Au même instant : **1 203** « J'aime » sur le serveur A, **1 205** sur le serveur B
E — *Eventually consistent* : sans nouvelle mise à jour, toutes les copies finissent par converger	Quelques secondes plus tard, les deux serveurs affichent **1 205**

À noter : BASE (D. Pritchett, 2008) est une **philosophie de conception**, pas une propriété obligatoire de toutes les bases NoSQL.

Réplication synchrone vs asynchrone



Synchrone : l'écriture n'est confirmée qu'une fois les copies à jour (cohérence immédiate, plus de latence). **Asynchrone** : réponse immédiate, copies à jour *à terme*.

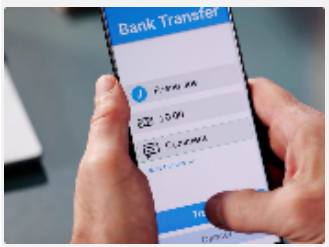
Ce sont des **mécanismes de réplication** : à eux seuls, ils ne font pas d'un système un système « ACID » ou « BASE ».

Pourquoi le modèle BASE ?

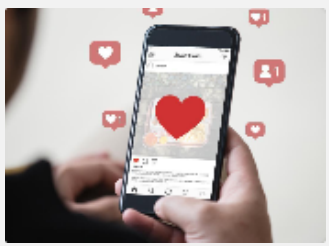
Le modèle BASE est **moins strict** : il accepte un compromis sur la **cohérence immédiate** pour privilégier :

- la **disponibilité immédiate** des données ;
- les **performances** sur de grands volumes de données (par exemple, effectuer très rapidement de nombreuses modifications sur beaucoup de données).

À retenir



ACID : la cohérence d'abord
→ banque, facturation, stocks.



BASE : la disponibilité d'abord
→ « J'aime », fils d'actualité, paniers.

Beaucoup de bases offrent **les deux** : MongoDB propose des transactions ACID ; Cassandra permet de régler la cohérence de chaque requête.

À vous de réfléchir : ACID ou BASE ?

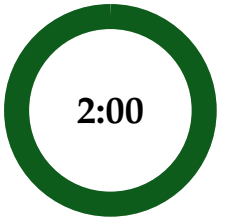
Question 7

Vous publiez une photo : votre ami ne la voit que 3 secondes plus tard. Est-ce grave ? Et s'il s'agissait du virement de votre salaire ?

Question 8 — Vote à main levée

Pour une application de **mobile money** (Bankily, Masrvi…), choisiriez-vous le modèle **ACID** ou **BASE** ?

Consigne : votez, puis débat de 2 minutes entre les deux camps.



cliquer pour lancer

La suite à la prochaine séance

Le théorème de CAP · Les familles de bases NoSQL